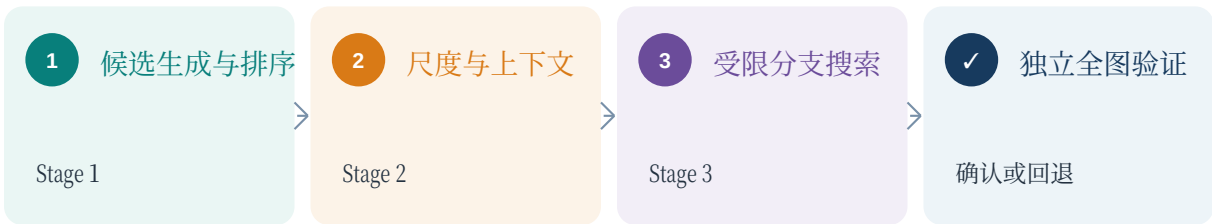


从候选排序到保守验证 分阶段自适应视觉搜索

一套面向高分辨率视觉问答的完整方法：先找到并排序值得看的区域，再补足尺度与上下文，在有限

一句话方法

Stage 1 负责“看哪里”，Stage 2 负责“怎样看”，Stage 3 负责“是否还要继续看”，独立验证层负责“新答案能否安全替换 P0”。



WHY

为什么需要分阶段视觉搜索

高分辨率图像的问题常常不是模型完全不会回答，而是关键物体太小、关系依赖周边场景，或一次裁剪没有同时保留足够多的上下文信息。

方法目标：把三个决策分开处理



继承 CVSearch 的部分

保留全图快速判断、问题中的目标提取、SAM 3 快速定位、失败后的语义区域树、CVSearch 初始区域排序，以及从深层到浅层的阈值搜索。它们仍是 Stage 1 的主体。

本文新增并评估的部分

在同一候选池内加入 Query 感知重排；把后续 Observation 明确拆为 Stage 2 与 Stage 3；用支持度、答案一致性和不确定性控制替换；最后加入候选无关的 Qwen2.5-VL-32B 全图验证。

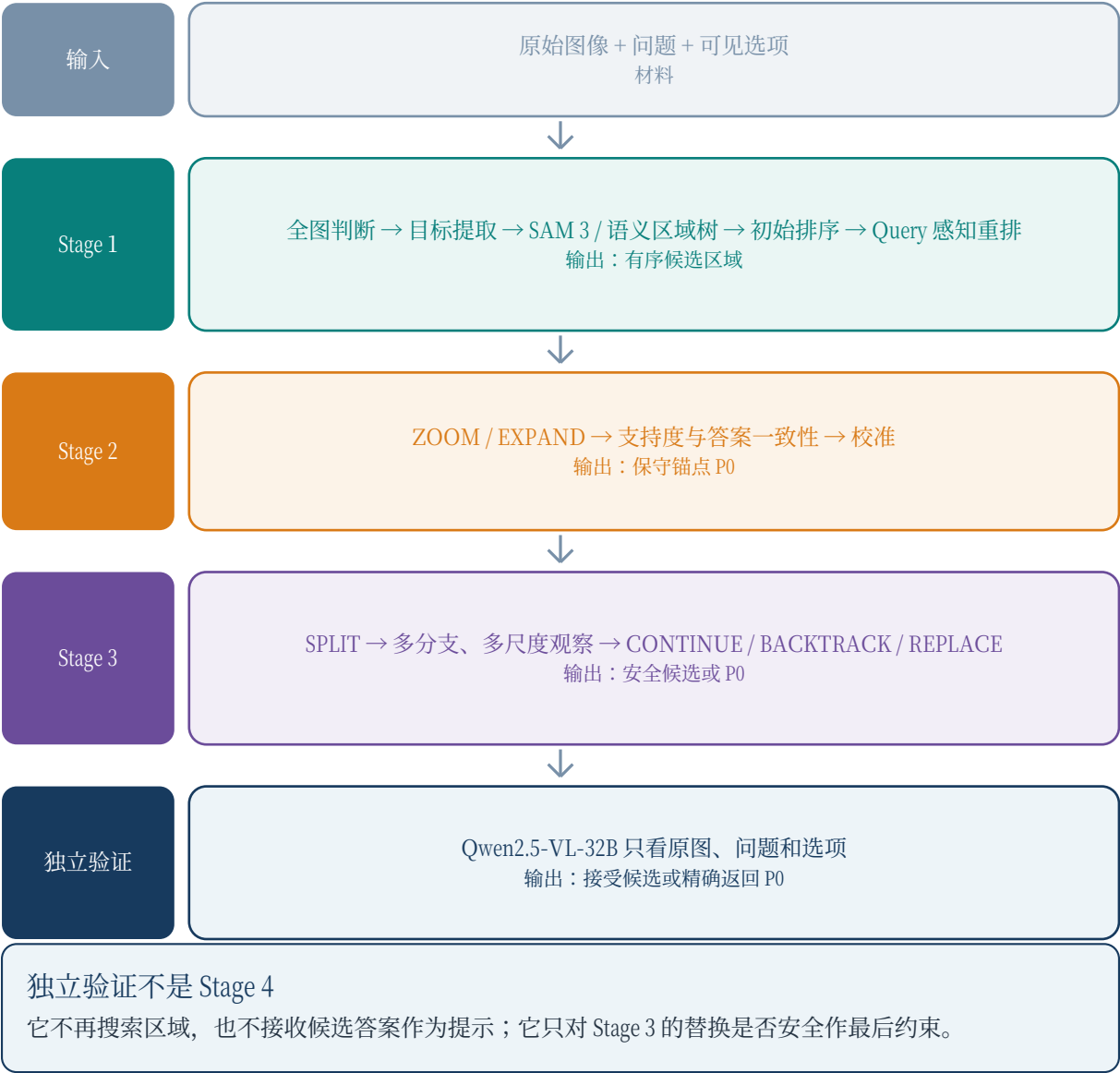
贡献边界

后文按阶段报告证据。Stage 1 的排序结果、Stage 2 的 Observation 修正和完整系统的端到端收益不会混为同一项提升。

PIPELINE

四部分各自解决一个问题

图像与问题只沿一条主链前进。每一部分都产生明确输出；后续部分可以补充证据，但无法安全确认时必须精确返回 Stage 2 的锚点 P0。



STAGE 1

全图足够时直接回答

Stage 1

先判断是否真的需要搜索。系统把整张图作为根节点，针对原问题估计回答置信度；只有当置信度没有越过配置的快速



需要搜索时，先把问题变成可定位的目标表达

多模态模型从问题中抽取关键视觉目标。实现会移除带代词、指代不清的表达，并把“all dogs”一类集合表达规范为可定位的目标词；如果没有生成有效目标，则由语言处理模块从问题中提取视觉对象。

SAM 3 是快速定位路径

SAM 3 结果	Stage 1 的处理	后续
全部目标定位成功	每个边界框直接变成 fast node	跳过语义树搜索
部分成功	保留已找到的 fast node	只为缺失目标搜索语义树
未找到目标	复用 SAM 3 backbone 特征	构建语义区域树

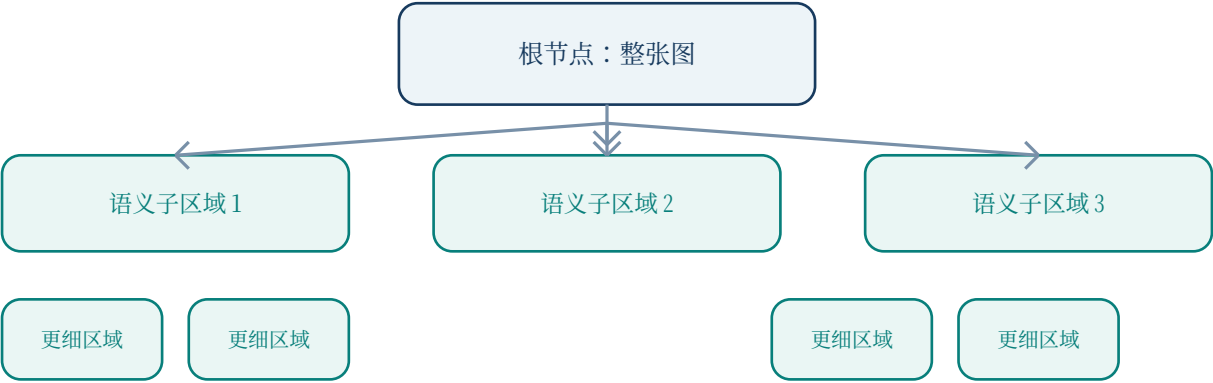
关键点

SAM 3
成功时负责“快”；失败并不终止流程，而是把同一次前向得到的视觉特征交给更细的区域搜索。

STAGE 1

SAM 失败后构建语义区域树

语义区域树不是按固定网格切图。它从 SAM 3 backbone 特征出发，把语义相近且空间相邻的区域逐层合并成候选，使后续搜索既能看局部，也能沿父子关系回到更完整的上



树是怎样得到的

步骤	实现中的做法	目的
视觉原子	SLIC 在特征图上生成约 600 个原子区域	保留局部边界
特征与连接	区域平均语义特征 + 归一化位置；建立邻接图	限制为相邻区域合并
选择分支数	连接约束的层次聚类，在 4 - 8 个子簇间选择	适应画面复杂度
选择标准	轮廓系数 - 1.5 × 边界框重叠代价	兼顾语义分离与少重叠

每个节点的复杂度来自区域内语义特征与平均方向的余弦差异，并按特征幅值缩放。复杂度不足时停止继续分裂；过于简单的子区域，多目标搜索最多访问 3 层。

为什么需要层级

小区域利于识别文字、颜色等细节；父区域保留人物—物体、左右和前后关系。树让两种尺度共享同一条搜索路径。

STAGE 1

CVSearch 先给出初始访问顺序

语义树产生的是候选集合，还需要决定先看哪一块。原始 CVSearch 用三个可解释量形成初始分数；Query 感知重排是在这一步之后执行，不会抹去这套基础顺序。

1

当前区域存在置信度

C_{current}：模型判断目标是否出现在 0.4 的节点不做这次存在性评估。

2

子区域聚合信息

C_{child}：取已评估子节点存在置信度

3

区域先验

P_{prior}：由树中相对复杂度、父节点置信度深度衰减

$$S_{cv} = 0.4 C_{current} + 0.4 C_{child} + 0.2 P_{prior}$$

存在置信度先从模型的 [-1, 1] 范围线性映射到 [0, 1]；同一层按 S_{cv} 从高到低排序。

最深层为什么只有两项

最深层没有可用的子区域信息，因此去掉 C_{child}，并把原来的 0.4 与 0.2 重新归一化：当前区域占 2/3，区域先验占 1/3。这样不会因为少一项而无故降低整层分数。

初始排序的职责

它回答“从树结构和目标存在性看，哪里值得先访问”。下一页的重排再回答“对当前原问题而言，哪里更相关”。

STAGE 1

Query 感知重排同时考虑问题和图像

对 CVSearch

已保留的每个候选裁剪，重排器计算四个量，并只在当前排序事件内转成百分位。这样不同模型分数尺度不会直接相加

百分位分量	来自哪里	它补充的信息
M：原问题 CLIP	候选裁剪 × 完整原问题	区域与真正任务是否匹配
A：定位表达 CLIP	至多 3 个最高增强查询分数的均值	目标词和问题改写是否指向该区域
C：节点复杂度	语义区域树节点	区域内部是否含可辨别结构
E：边缘密度	裁剪灰度的纵横像素差	文字、轮廓等局部变化是否丰富

$$R = \beta M + (1 - \beta)A \quad V = \lambda C + (1 - \lambda)E$$

R 是问题相关性；V 是视觉信息量。最终排序分数为 $S_{\text{query}} = \alpha_{\text{eff}} R + (1 - \alpha_{\text{eff}}) V$ 。

最终盲测版本使用 $\beta=0.60$ 、 $\alpha=0.65$ 、 $\lambda=0.70$ 。排序使用稳定降序；分数相同时保留候选原有相对次序。

问题自适应权重只看问题文本

问题信号	权重变化	直观含义
细节需求：颜色、文字、材质等	α_{eff} 减 0.15	适当增加视觉信息量的作用
上下文需求：左右、前后、相邻等	α_{eff} 加 0.15	更强调与完整问题的语义关系
上下文但无外观描述	λ_{eff} 最多减 0.25	边缘信息获得更多权重
含颜色 / wearing / holding 等外观词	解除上述 λ 折减	保留复杂度对外观区域的作用

所有权重都截断到 $[0, 1]$ ；这些规则不读取正确答案、目标框或数据集身份。

STAGE 1

按层访问把排序转化为实际观察

排序完成后，系统不是同时把所有裁剪交给模型，而是按层、按候选逐个访问。每次访问都针对问题计算回答置信度，

1

从最深允许层开始

单目标最多第 2 层，多目标最多第 3 层；失败后回到更浅层。

2

层内先初排再重排

CVSearch 初始分数给出基础顺序，Query 感知分数在同一候选池内重排。

3

逐候选计算回答置信度

超过当前阈值即可返回；最深层可启用父区域存在性检查。

4

到检查点后降低阈值

默认下降序列为 0.05、0.10、0.20，最低不越过下界，并复查已访问候选。

5

仍未命中则回到浅层

第 1 层只访问排序最高的节点；完全失败时保留排序后的候选作为回退材料。

Top-3 不是剪枝

实现保留所有候选的身份集合；Top-3

用于衡量前部排序质量和限制评测预算，不表示把其余区域从搜索状态中删除。

Stage 1 输出

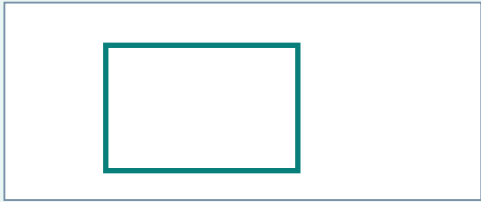
有序候选区域、原图坐标、层级与树范围、初始/重排分数、已访问节点及回答置信度。Stage 2 在这些可追溯材料上选择观察尺度。

STAGE 2

ZOOM 补细节，EXPAND 补上下文

Stage 1 已经回答“先看哪里”，但同一个候选框不一定提供正确尺度。Stage 2 从当前答案和其原生裁剪出发，分别构造更紧的细节视图与额外的上下文视图，再比较证据是否真的变充分。

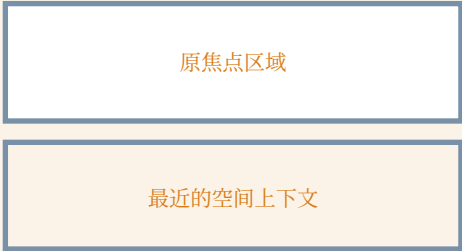
ZOOM：缩小视野



P0 原生区域 → 更紧的坐标裁剪

候选裁剪必须严格小于当前裁剪；按原图坐标重渲染，提升小目标区域的可观察性。

EXPAND：增加邻近区域



原焦点区域

最近的空间上下文

保持 P0 焦点顺序，在下方追加一个最近的原生 CVSearch 区域；新增区域必须围绕焦点之外的面积。

比较	ZOOM	EXPAND
主要补什么	局部细节不足	关系与场景上下文不足
怎样改变输入	换成更紧的原图裁剪	保留焦点并追加独立上下文面板
是否自动改答案	否：只产生候选观察	否：只产生候选观察

为什么两种观察都要保留

只放大可能看清目标却丢掉方位；只扩展可能保留关系却让细节再次变小。答案发生变化时，Stage 2 要求两类动作都已有可用观察，避免单一尺度独断。

STAGE 2

Stage 2 把可回退答案保存为 P0

每个 ZOOM / EXPAND

观察都接受两次分开的检查：一是画面是否足以支持回答问题，二是同一观察得到的答案是否稳定。选择器只看这些

$$\text{raw support} = P(\text{sufficient}) \times \sqrt{\text{answer consistency}}$$

P(sufficient) 来自答案无关的 Yes/No 支持度提示；实现中的 support consistency 固定为 1，因此原始支持度只再乘答案一致性的平方根。

$$\text{support uncertainty} = 1 - P(\text{sufficient})$$

这就是仍然保留的不确定性之一：观察越不能证明证据充分，支持不确定性越高。

从原始支持度到可比较概率

量	怎样得到	在选择中的作用
原始支持度	当前视图与候选视图各自独立打分	计算观察前后的原始增益
校准支持度	用冻结的保序校准映射 raw support	校正不同分数段的可信程度
答案一致性	同一观察的聚合答案出现频率	低于 0.8 时不允许选择
支持度增益	候选校准支持 - 当前校准支持	通常至少增加 0.05；平台期需有原始进步

P0 是精确可重建的保守锚点

Stage 2

在候选通过支持度、增益、一致性和跨动作冲突检查时选用该答案，否则保留输入答案。随后把最终 Stage 2 输出、原始模型响应、产生它的节点键与支持视图冻结为 P0，供 Stage 3 随时精确回退。

保守不是拒绝改进

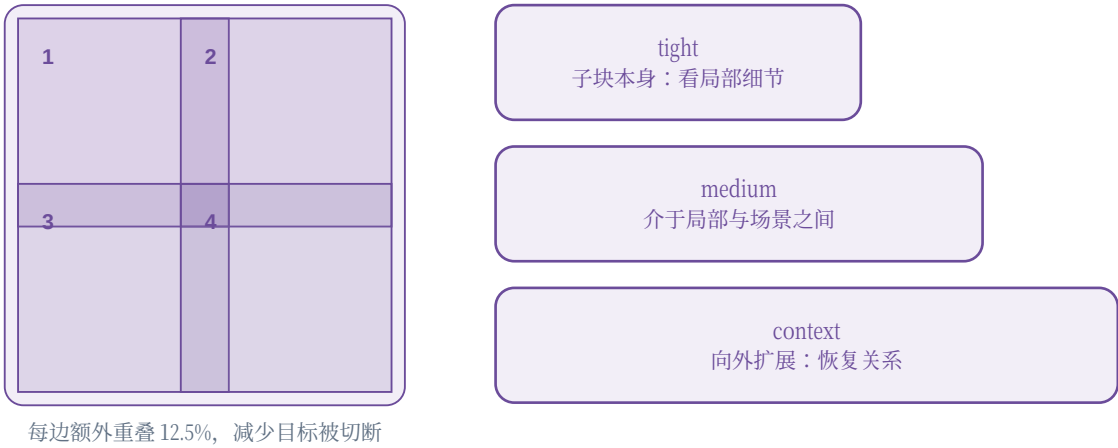
P0

允许后续提出新答案，但要求每次失败、证据冲突、预算耗尽或材料异常都返回完全相同的已冻结输出，而不是重新调用模

STAGE 3

SPLIT 只在局部位置仍模糊时继续细分

当问题需要的细节仍无法定位，SPLIT
在候选区域内部建立有限分支。它不重新搜索整张图，也不允许无限递归：每个节点固定分成2×2个带重叠的子块，最大深度为2。



先排序分支，再按固定预算观察

同级子块先按问题相关性、特征偏离与边缘密度排序：相关性占 0.70；其余 0.30
为两种视觉信息的等权组合。最终策略最多保留 6 个已排序分支，并从 tight、medium、context
三种原生坐标渲染中形成观察序列。

约束	冻结设置	作用
空间分支	2×2、12.5% 重叠、深度 2	局部化但不无限展开
观察预算	最终策略固定读取 8 个观察	计算可控，分支之间可回退
结构门槛	至少 2 个不同渲染支持同一答案	避免单张裁剪偶然改答

多尺度不是简单重复
tight、medium、context 的原图坐标与渲染哈希必须不同；相同画面重复回答不能充当两份独立支持。

STAGE 3

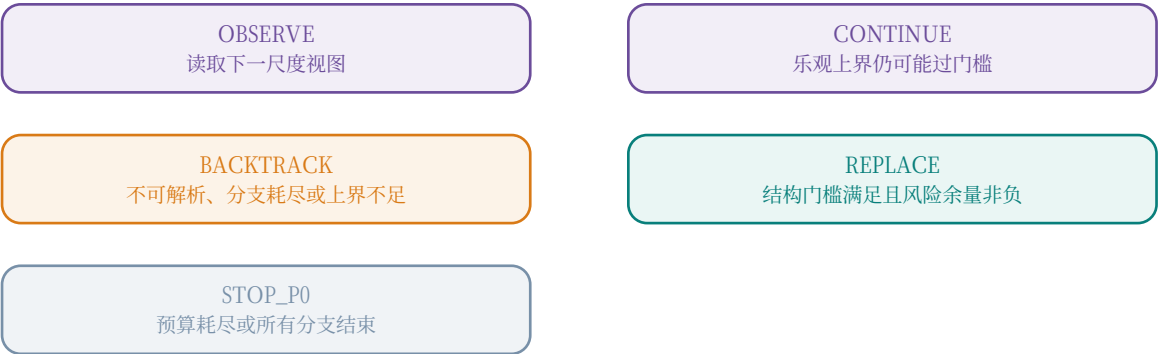
不确定性控制决定继续、回退或提出替换

原来的 uncertainty 仍然存在，而且是 Stage 3 的核心输入。它不直接等于模型自报置信度，而是与多视图一致性、最低支持度、相对 P0 的支持增益和冲突强度一起进入风险校准。

风险特征	回答的问题
P0 answer uncertainty	当前保守答案本身有多不稳定？
agreement	已解析观察中，有多少支持同一个新答案？
minimum support	支持新答案的最弱视图是否仍可靠？
support gain	新答案相对 P0 的支持度是否真正增加？
P0 conflict margin	支持新答案的证据是否明显强于仍支持 P0 的竞争证据？

校准器分别估计期望收益与破坏风险，并使用“收益 - 2×风险”的保守余量。最终策略在第 8 个观察处决策，且至少需要 2 个一致视图。

状态转移只允许五种可审计动作



FALLBACK_P0：材料异常时关闭失败路径
校准缺失、输入哈希或阶段材料不一致时，不进入普通状态机，直接精确重建 Stage 2 输出。它与正常预算结束时的 STOP_P0 都返回同一个 P0。

uncertainty 与独立验证的分工
uncertainty 判断 Stage 3 候选是否值得提出；下一页的 32B 全图答案检查该候选是否遇到独立的高置信冲突。前者没有被后者替代。

VERIFY

独立全图答案负责最后的否决与确认

验证模型是 Qwen2.5-VL-32B。它在独立调用中只接收完整原图、原问题和可见选项，最大处理像素为 4,194,304；提示中不出现 P0、Stage 3 候选、裁剪框、数据集身份或正确答案。



RESULTS

Qwen 上的提升来自不同阶段

下面四行分别回答排序是否更好、盲测版本是否守住结果、尺度观察能否安全修正，以及完整开发策略能否在零破坏

阶段与材料	Qwen 结果	可以支持的结论
Stage 1 · 已打开开发集 30 题；21 题进入排序	R@3 85.71% → 95.24% 准确率 80.00% → 83.33%	Query 感知排序在该开发样本上改善候选前部质量。
Stage 1 · v6 混合盲测 20 题；14 题进入排序	R@3 78.57% → 78.57% 准确率 85.00% → 85.00%	最终 Stage 1 版本在 R@3 与答案准确率上无回退。
Stage 2 · 预声明 Observation 测试	V* 17/20 → 18/20 TreeBench 5/12 → 6/12	ZOOM / EXPAND 各测试单元合计修正 2 题，HR 4K/8K 不下降。
Stage 2+3+验证 · 接受的开发策略 224 题；512 个计分单元	Qwen 净增 +7 全体净增 +16，0 次破坏	两种 backbone 都为正；在已打开开发与 validation_v3 上零破坏。

需要同时披露的细节

开发排序的 Recall@1 为 42.86% → 52.38%；但 v6 混合盲测的 Recall@1 从 57.14% 降到 42.86%。因此最终主张聚焦预先设定的 R@3 与答案准确率，不宣称首位排序全面改善。

如何读这些数字

Stage 1 的结果衡量候选排序；Stage 2 的结果衡量尺度观察；+7 来自加入 Stage 3 与独立验证后的接受开发策略。开发数据已打开，只有下一页的一次封存 MME 结果用于完整系统端到端结论。

END-TO-END

MME-RealWorld-Lite：完整系统净增 27 题

封存评测只执行一次。这里比较 Qwen 基线与包含 Stage 1、Stage 2、Stage 3 和独立验证层的完整系统；因此 +27 是端到端收益，不能归到任一单独阶段。

Qwen · 正确题数

880/1919 → 907/1919 +27

45.86% → 47.26% (+1.41 个百分点) · 49 次修正 · 22 次破坏

选择替换 99 题；平均每题 8.41 个观察；运行失败 0。

跨 backbone 补充

InternVL 从 879/1919 提升到 897/1919，净增 +18；两种 backbone 合计净增 +45。

局限

1	封存评测只有一次	支持这次端到端比较，但不能替代多次独立复验。
2	验证层需要额外计算	Qwen2.5-VL-32B 全图调用与平均 8.41 个观察增加推理成本。
3	端到端结果不可单阶段归因	需要新的封存消融，才能分离 Stage 1/2/3 与验证层的独立贡献。

结论

这套方法把“找到区域”“补足证据”“控制不确定性”和“安全替换答案”拆成四个可审计环节。Qwen 上的结果表明：排序改进可以安全进入后续观察，保守锚点能限制破坏，而候选无关的全图验证能在真实高分辨率评测中提升。

最终要点

提升不是来自更激进地改答案，而是来自更清楚地规定：何时看局部、何时补上下文、何时继续、何时相信新答案，以及如何避免 P0。